https://www.turing.org.uk/turing/pi2/run.jpg

**HW 8 coming soon.**

**Quiz 11 coming soon.**
**Midterm 3 next Tue 9:30am, Apr 14.**

# Class 22:
# Class P

University of Virginia
CS3120: DMT2
https://weikailin.github.io/cs3120-toc
Wei-Kai Lin

# Plan

**Complexity**

*Running time / space*

*Class* **P**

*Problems not in* **P**

Textbook [TCS] Section 12

https://introtcs.org/public/lec_10_efficient_alg.html
Efficient Computation

# Recap: Computability

# Recap: Rice's Theorem (9.15)

If $F$ is a sematic property, then either $F$ is **uncomputable**, or $F$ is trivial.

By definition, $F$ is trivial iff for all Turing machine $M$,

$$F(M) = 0$$
$$F(M) = 1$$

$F$ is sematic if $\forall\, M_1, M_2$ s.t. $M_1 \equiv M_2 \Rightarrow F(M_1) = F(M_2)$

# Recap: Busy Beaver

$BB(n)$: the TM such that among all $n$-state and 2-alphabet Turing machines that halt on no input, $BB(n)$ runs longest.

Well-defined, uncomputable!

A *Turing Machine,* is defined by $(\Sigma, C, \delta)$:
$C \in \mathbb{N}$: a finite number of states
$\Sigma$ : alphabet $-$ finite set of symbols
$\Sigma \supseteq \{0, 1, \triangleright, \emptyset\}$
$\boldsymbol{\delta}$: transition function
$\boldsymbol{\delta}: [C] \times \Sigma \to [C] \times \Sigma \times \{\mathbf{L}, \mathbf{R}, \mathbf{S}, \mathbf{H}\}$

Any Turing Machine $(\Sigma, n, \delta)$ consists of $n$ state.

Quantamagazine

https://www.quantamagazine.org/amateur-mathematicians-find-fifth-busy-beaver-turing-machine-20240702/

Today, the team declared victory. They've finally verified the true value of a number called BB(5), which quantifies just how busy that fifth beaver is. They obtained the result — 47,176,870 — using a piece of software called the Coq proof assistant, which certifies that mathematical proofs are free of errors.

# Module 4: Complexity
# (P vs NP)

# *How have we considered "reasonable cost" so far?*

# Cost-Sensitive Computing

| Model of Computation | What to Count | Complexity Classes and Results |
|---|---|---|
| Boolean Circuits / NAND-CIRC programs | # gates / depth | |
| DFA / NFA / Regexp | | |
| ? (Algorithms course) | time? what's unit "time"? | |
| Turing Machines | | |

# Cost-Sensitive Computing

| Model of Computation | What to Count | Complexity Classes and Results |
|---|---|---|
| **Boolean Circuits / NAND-CIRC programs** | Gates in Circuit<br>Lines in NAND-CIRC program | $SIZE_{n,m}(s)$<br>Size Hierarchy Theorem |
| **DFA / NFA / Regexp** | Number of States<br>Length of Regexp | NFA with $n$ states ~ DFA with $2^n$<br>length of reg exp |
| **?**<br>**(Algorithms course)** | Constant time-ish operations (?) | $Sorting \in \Theta(n \log n)$<br>(number of comparisons to comparison-sort $n$ items) |
| **Turing Machines** | # steps to halt<br># tape cells | |

# Costs of Turing Machines

**Length of description** (num of states?)

**Amount of tape needed** (“Space Complexity”)

**Number of steps before halting**
(“Running Time Complexity”)

# Some ~~Turing Machines~~

Functions | Time

Binary addition
n-bit input, time: $ADD(\underbrace{a}_{n}, \underbrace{b}_{n\text{-bit}})$ $O(n)$?

$\exists$ TM ... alg

Binary multiplication $MUL(a, b)$ $\tilde{O}(n \log n)$?
n-bit input, time:

Busy Beaver? $BB(n)$ X

# Relating Time and Space

$T =$

If a TM running on an $n$-length input executes in $23(n^2)!$ steps, what can we say about its space complexity?

$\text{Space} \overset{?}{\leq} T$

$\overset{?}{\cancel{\geq}}$

# Relating Space and Time

If a TM running on an $n$-length input executes in $\underbrace{\Theta(n)}_{S}$ space, what can we say about its running time?

$S \leq \text{Time} \leq \{\text{finite}\}$

$BB(\#\text{states})$

$\leq 2^{O(S)}$

# 17

# *Space bounded computation*

PLAN: Example of space bounded algorithms, importance of preserving space. The classes L and PSPACE, space hierarchy theorem, PSPACE=NPSPACE, constant space = regular languages.

Fall 6210
Grad
Theory Comp
(Complexity)

We will not talk more about space complexity, but it is related to many topics in this course.

# Time Complexity Classes

# Defining TIME Complexity Classes

Definition (13.1):
$TIME_{\text{TM}}(T(n))$ is the set of functions for which a Turing Machine $M$ exists such that $M$ halts after at most $T(n)$ steps for all $n$-bit input and $M$ computes the function.

**Definition 13.1 (Running time (Turing Machines))**

Let $T : \mathbb{N} \to \mathbb{N}$ be some function mapping natural numbers to natural numbers. We say that a function $F : \{0,1\}^* \to \{0,1\}^*$ is *computable in $T(n)$ Turing-Machine time (TM-time for short)* if there exists a Turing machine $M$ such that for every sufficiently large $n$ and every $x \in \{0,1\}^n$, when given input $x$, the machine $M$ halts after executing at most $T(n)$ steps and outputs $F(x)$.

We define $TIME_{\text{TM}}(T(n))$ to be the set of Boolean functions (functions mapping $\{0,1\}^*$ to $\{0,1\}$) that are computable in $T(n)$ TM time.

$TIME_{\text{TM}}(T(n))$ is the set of functions for which a Turing Machine $M$ exists such that $M$ halts after at most $T(n)$ steps for all $n$-bit input and $M$ computes the function.

# What functions are in $TIME_{\text{TM}}(28)$?

$F(x) = 0$

$M(x) =$ 1. ignore input

2. write 0 output

$TIME_{\text{TM}}(T(n))$ is the set of functions for which a Turing Machine $M$ exists such that $M$ halts after at most $T(n)$ steps for all $n$-bit input and $M$ computes the function.

# What functions are in $TIME_{\text{TM}}(28)$?

**Definition.** The execution of a TM, $M = (\Sigma, C, \delta)$ on input $x \in \{0,1\}^*$ is this process:

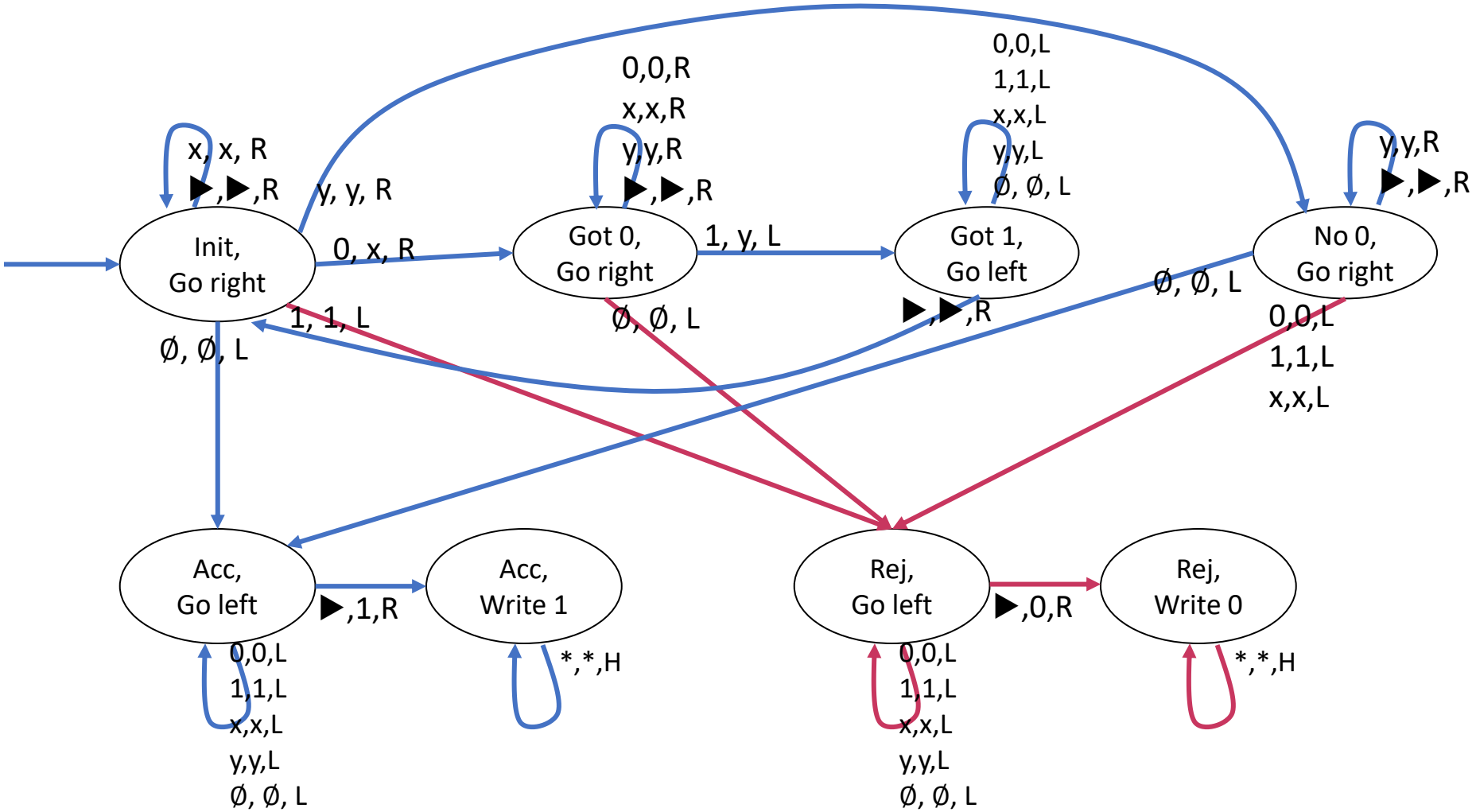
1. Initialize $T$ as $\triangleright, x_0, x_1, \dots, x_{n-1}, \emptyset, \emptyset, \emptyset, \dots$ where $n = |x|$.

   That is, $T[0] = \triangleright$, $T[i+1] = x_i$ for $i \in [n]$, and $T[i] = \emptyset$ for $i > n$.
2. Initialize two natural number variables, $i = 0, s = 0$.
3. **repeat**
   1. $(s', \sigma', D) = \boldsymbol{\delta}(s, T[i])$
   2. $s := s', T[i] := \sigma'$
   3. if $D$ = **R**: $i := i + 1$
      if $D$ = **L**: $i := \max\{i-1, 0\}$
      if $D$ = **H**: **break**
4. If the process finishes (the repeat breaks), the result of this process is: $M(x) = T[1], \dots, T[m]$ where $m > 0$ is the smallest integer, $T[m+1] \notin \{0,1\}$.

$TIME_{\text{TM}}(T(n))$ is the set of functions for which a Turing Machine $M$ exists such that $M$ halts after at most $T(n)$ steps for all $n$-bit input and $M$ computes the function.

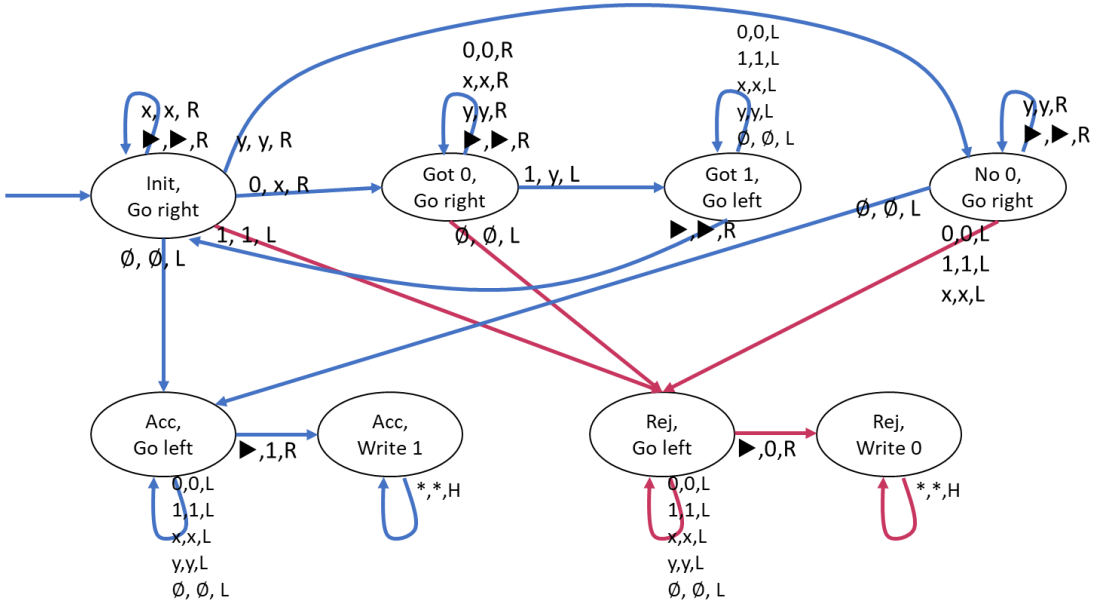
# Is the 0n1n machine in $TIME_{\text{TM}}(17n^4)$?

0n1n machine, HW6



$TIME_{\text{TM}}(T(n))$ is the set of functions for which a Turing Machine $M$ exists such that $M$ halts after at most $T(n)$ steps for all $n$-bit input and $M$ computes the function.

# Is the 0n1n machine in $TIME_{\text{TM}}(17n^4)$?



$\in EVEN$ ?

No, but this is a trick question like asking if there is an Orange in the set.

$TIME_{\text{TM}}(17n^4)$ is a set of **(Boolean) functions**. It is about whether there exists a TM that can compute a function within $17n^4$ steps (where $n$ is the size of the input).

$TIME_{\text{TM}}(T(n))$ is the set of functions for which a Turing Machine $M$ exists such that $M$ halts after at most $T(n)$ steps for all $n$-bit input and $M$ computes the function.

# Is the 0n1n function in $TIME_{\text{TM}}(17n^4)$?

Yes Yes

Is the 0n1n function in $TIME_{\text{TM}}(17n^4)$?

halts in $\frac{O(n^2)}{8 \cdot n^2}$

$TIME_{\text{TM}}(T(n))$ is the set of functions for which a Turing Machine $M$ exists such that $M$ halts after at most $T(n)$ steps for all $n$-bit input and $M$ computes the function.

# Is the 0n1n function in $TIME_{\text{TM}}(17n^4)$?

Is the 0n1n function in $TIME_{\text{TM}}(17n^4)$?

"Not yet" – if this machine can compute it for all length $n$ inputs in $17n^4$ steps (where $n$ is the size of the input) that would be a proof that $0n1n \in TIME_{\text{TM}}(17n^4)$

# Class P

# Definition: Complexity Class: P

A class for "Polynomial Time"

Class **P** = $\bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$

any polynomial time

Note: TCS book definition is **P** = $\bigcup_{c \in \{1,2,3,\ldots\}} TIME_{TM}(n^c)$

# P is Robust to Machine Definitions

$\mathbf{P}_{TM} = \bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$ One-way infinite tape TM

$\|$

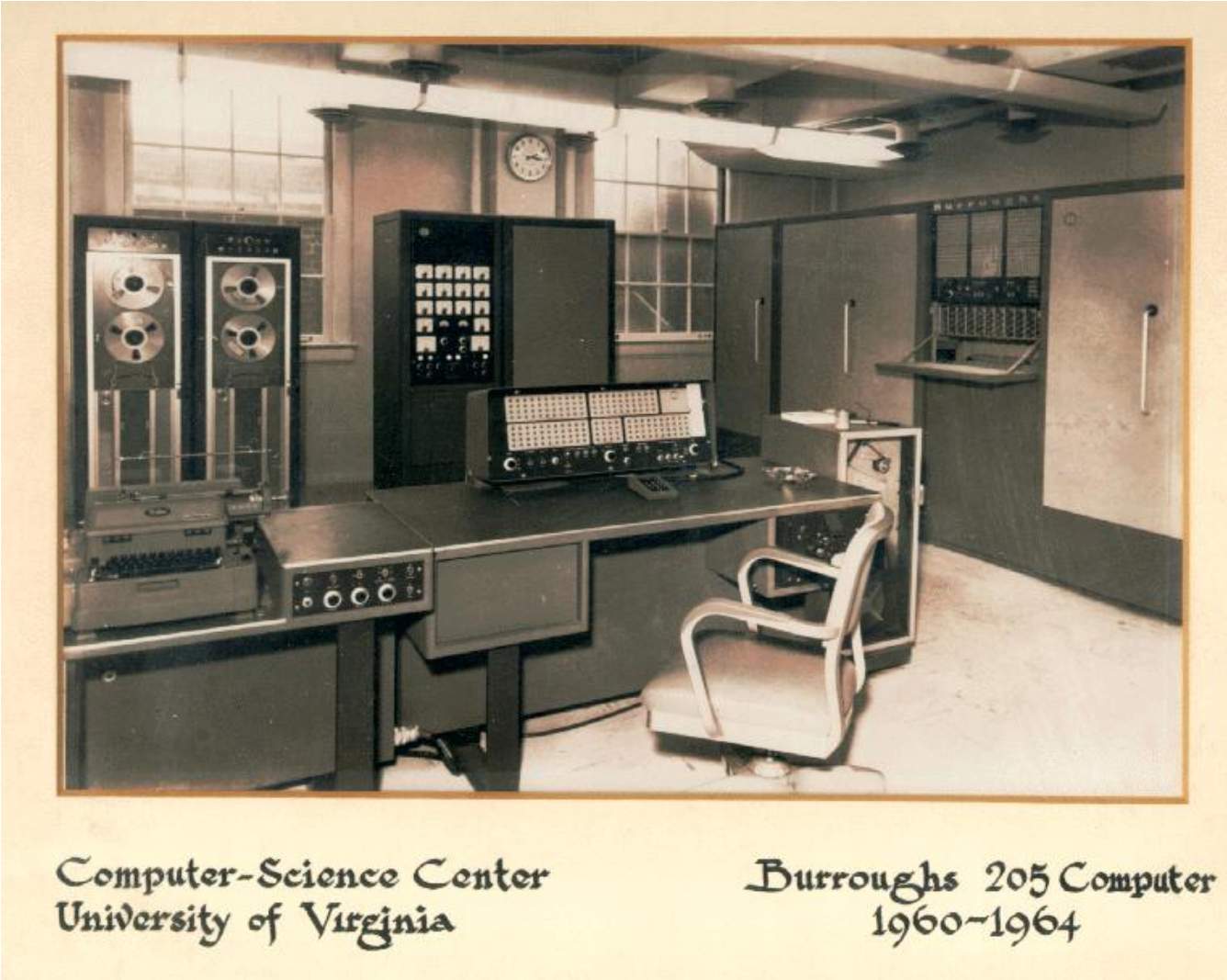
$\mathbf{P}_{TM2w} = \bigcup_{c \in \mathbb{N}} TIME_{TM2w}(n^c)$ Two-way infinite tape TM

$\|$

$\mathbf{P}_{TM\times 2} = \bigcup_{c \in \mathbb{N}} TIME_{TM\times 2}(n^c)$ Two-tape TM
(with a read/write head on each tape)

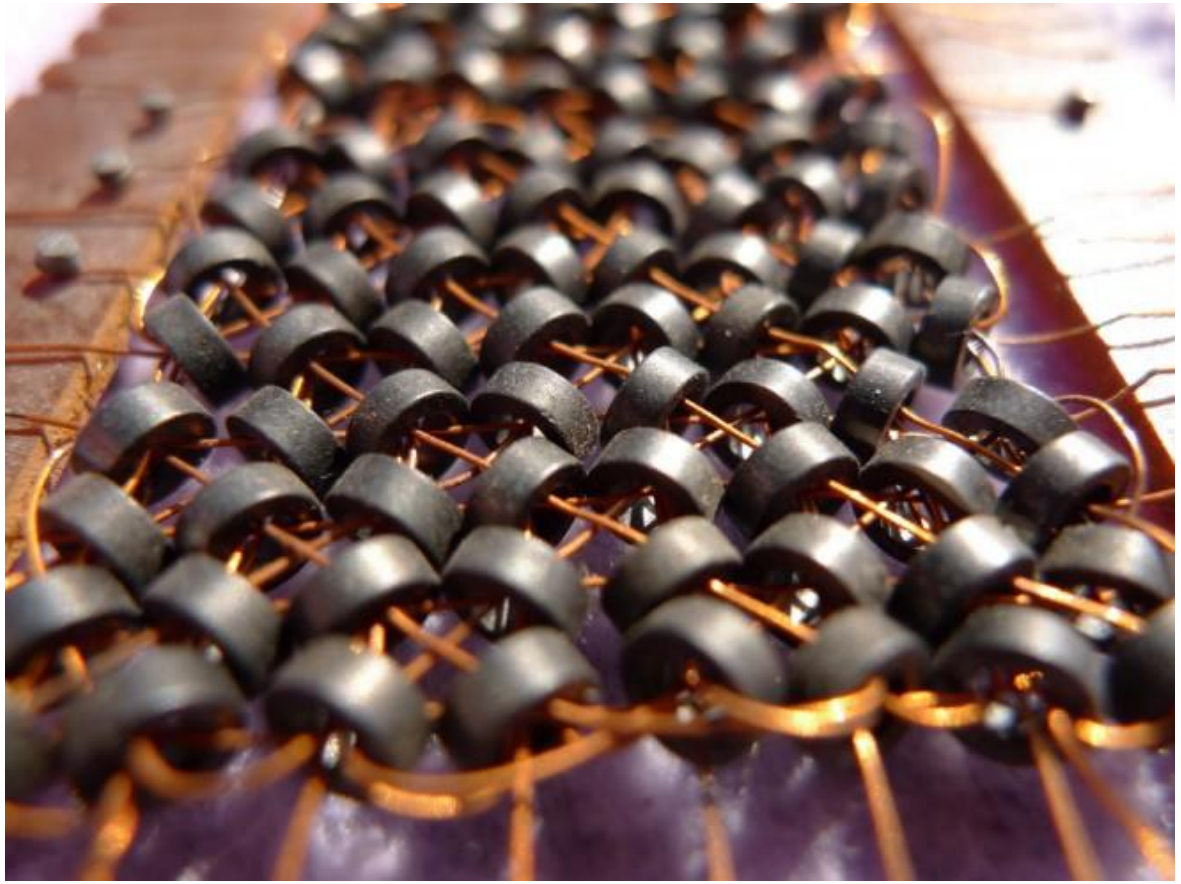
simulate in poly(n) extra time

# Tape-Based Memory

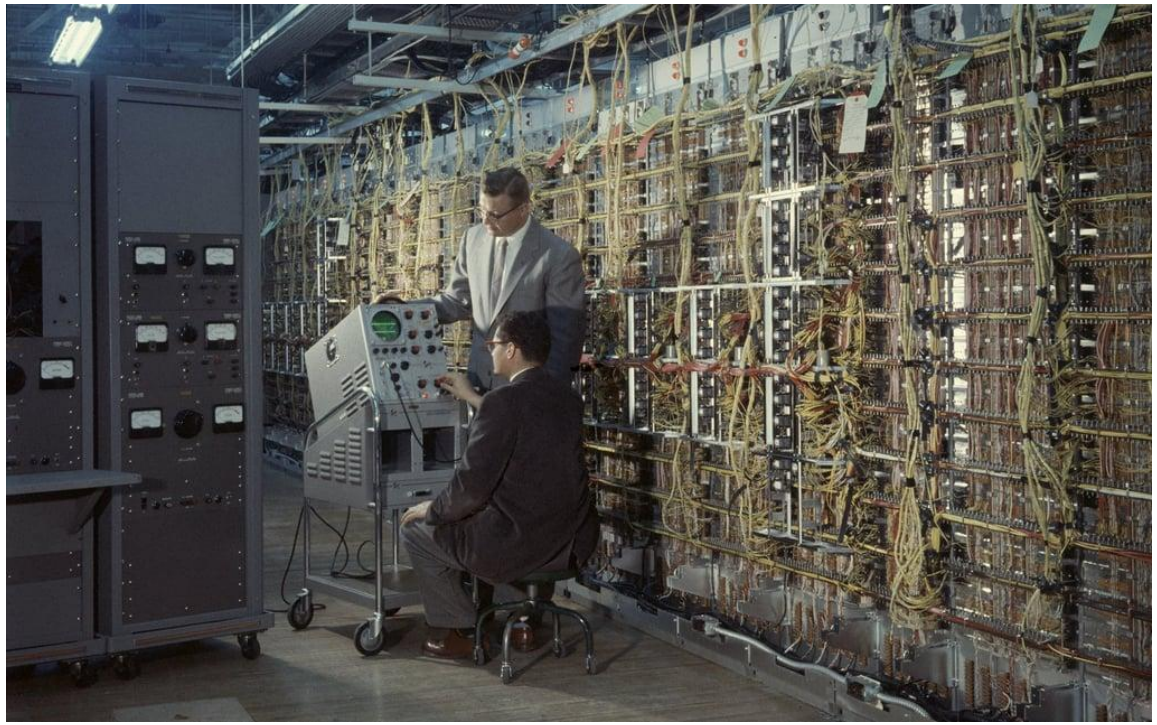


A tape was a reasonable memory model for practical computers until memories got big

# Random-Access Memory



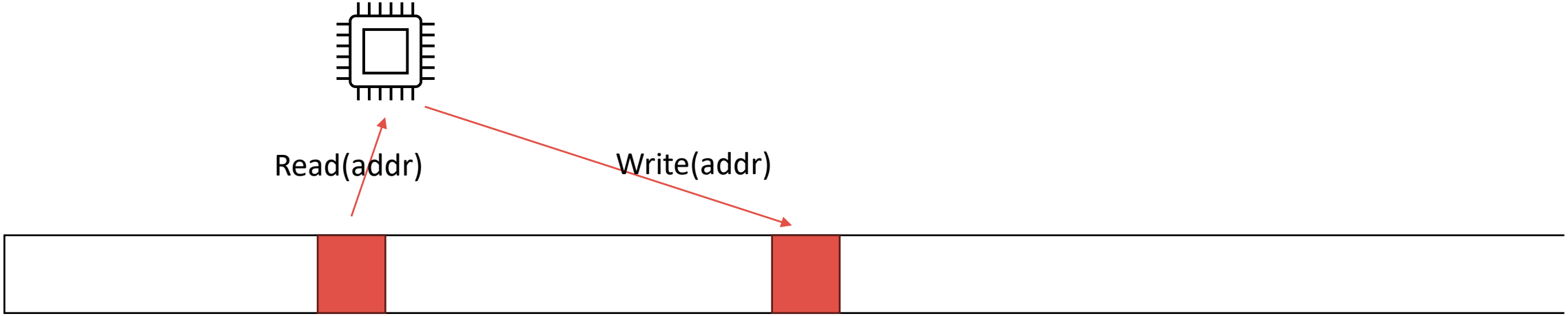
Magnetic Core Memory (Jay Forrester, 1951)
MIT tried to license patent for 2 cents/bit!



**Semi-Automatic Ground Environment**
FSQ-7 memory: 65,536 ($2^{16}$) 32-bit words

# Random-Access Memory (RAM) Machines

- **Unbounded memory array**, can be read and written using natural number indices
- **CPU:**
  1. read or write an array “word” by index/address
  2. compute any function on constant number of “words” (registers)




*Is a RAM machine equivalent (in computing power) to a TM?*

# RAM Machine Complexity

- **Unbounded memory array**, can be read and written using natural number indices
- **CPU:**
  1. read or write an array "word" by index/address
  2. compute any function on constant number of "words" (registers)

$$\mathbf{P}_{\mathrm{RAM}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{RAM}}(n^c)$$

*Is* $\mathbf{P}_{\mathrm{RAM}} = \mathbf{P}_{\mathrm{TM}}$*?*

$$\mathbf{P}_{\mathrm{RAM}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{RAM}}(n^c)$$

$$\mathbf{P}_{\mathrm{RAM}} = \mathbf{P}_{\mathrm{TM}\times 4} = \mathbf{P}_{\mathrm{TM}}$$

We can simulate RAM memory with a multi-tape TM:

- Have a second tape to keep track of current location (increment each time we move right, decrement for left)
- Have a third tape to record the target location
- Move until the two tapes match (maybe need another tape?)

Details not important (only impact $c$) – but if you don't like this sketchy argument see TCS 8.2 ("The gory details (optional)")

- Whatever a RAM machine can compute in $T(n)$ steps, a tape TM can compute $T(n)^4$ time (Theorem 13.5)

We expect you to understand the crux of this argument, but not the details which we won't cover

# P is Robust to Machine Definitions

$\mathbf{P}_{TM} = \bigcup_{c\in\mathbb{N}} TIME_{TM}(n^c)$ One-way infinite tape TM

$\mathbf{P}_{TM2w} = \bigcup_{c\in\mathbb{N}} TIME_{TM2w}(n^c)$ Two-way infinite tape TM

$\mathbf{P}_{\mathrm{RAM}} = \bigcup_{c\in\mathbb{N}} TIME_{\mathrm{RAM}}(n^c)$ **RAM** machine

$O(T^3) \Leftarrow T$ if $T = n^c \Rightarrow O(T^3) = O(n^{3c})$

$$\mathbf{P}_{TM} = \mathbf{P}_{TM2w} = \mathbf{P}_{\mathrm{RAM}}$$

# P is Robust to Machine Definitions

$\mathbf{P}_{TM} = \bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$ One-way infinite tape TM

$\mathbf{P}_{TM2w} = \bigcup_{c \in \mathbb{N}} TIME_{TM2w}(n^c)$ Two-way infinite tape TM

$\mathbf{P}_{\mathrm{RAM}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{RAM}}(n^c)$ **RAM** machine

$\mathbf{P}_{\mathrm{Python}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{Python}}(n^c)$ Idealized **Python** interpreter

$$\mathbf{P}_{TM} = \mathbf{P}_{TM2w} = \mathbf{P}_{\mathrm{RAM}} \stackrel{?}{=} \mathbf{P}_{\mathrm{Python}}$$

# P is Robust to Machine Definitions

$\mathbf{P}_{TM} = \bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$ One-way infinite tape TM

$\mathbf{P}_{TM2w} = \bigcup_{c \in \mathbb{N}} TIME_{TM2w}(n^c)$ Two-way infinite tape TM

$\mathbf{P}_{\mathrm{RAM}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{RAM}}(n^c)$ **RAM** machine

$\mathbf{P}_{\mathrm{Python}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{Python}}(n^c)$ Idealized **Python** interpreter

$$\mathbf{P}_{TM} = \mathbf{P}_{TM2w} = \mathbf{P}_{\mathrm{RAM}} = \mathbf{P}_{\mathrm{Python}}$$

# P is Robust to (Most) Machine Definitions

$\mathbf{P}_{TM} = \bigcup_{c \in \mathbb{N}} TIME_{TM}(n^c)$ One-way infinite tape TM

$\mathbf{P}_{TM2w} = \bigcup_{c \in \mathbb{N}} TIME_{TM2w}(n^c)$ Two-way infinite tape TM

$\mathbf{P}_{\mathrm{RAM}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{RAM}}(n^c)$

**RAM** machine
- READ(i), WRITE(i, val), ADD(a,b), MULT(a,b)
- JUMP(k)

$\mathbf{P}_{\mathrm{Python}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{Python}}(n^c)$ Idealized **Python** interpreter

$\mathbf{P}_{\mathrm{QC}} = \bigcup_{c \in \mathbb{N}} TIME_{\mathrm{QC}}(n^c)$ Quantum Computer

$$\mathbf{P}_{TM} = \mathbf{P}_{TM2w} = \mathbf{P}_{\mathrm{RAM}} = \mathbf{P}_{\mathrm{Python}} \stackrel{?}{=} \mathbf{P}_{\mathrm{QC}}$$

# Logistics

~~1:30~~

Thu 2:30 Mead coffee.

Complexity

HW 8 coming soon.

Quiz 11 coming soon.

# Plan

**Complexity**

*Problems not in* **P**

Textbook [TCS] Section 12

https://introtcs.org/public/lec_10_efficient_alg.html
Efficient computation